RAG: Dynamic Parsing & Dynamic Chunking

Smart Document Assistant

Abstract

This project is a document assistant prototype. Users upload PDF, DOCX, or TXT files. The application extracts text, creates chunks, compares a question with chunks using TF-IDF and cosine similarity, and displays matching passages with source information. This version demonstrates ingestion, parsing, chunking, and retrieval; it does not use a hosted LLM to generate new answers.

Problem Statement

Long documents can contain paragraphs, headings, lists, tables, and different layouts. Poor chunk boundaries can separate related information. A retrieval system should locate relevant passages and provide source context so users can verify evidence.

Objectives

- Support PDF, DOCX, and TXT uploads.
- Extract text from supported file types.
- Create smaller chunks with overlap.
- Rank chunks against a question using TF-IDF and cosine similarity.
- Display relevant passages and available source information.

RAG Pipeline

A typical RAG workflow is Ingest

- Parse
- Chunk
- Index
- Retrieve
- Generate.

Ingestion receives the document. Parsing extracts text and structure. Chunking creates retrieval units. Indexing creates searchable representations. Retrieval selects relevant

passages. Generation can use those passages as context for an LLM. In this prototype, TF-IDF is used for indexing/retrieval and the output is passages rather than an LLM-generated answer.

Dynamic Parsing

Parsing adapts extraction to the file type. This prototype extracts PDF text page by page, DOCX paragraphs, and TXT plain text. Source information such as page or section helps trace results. Scanned PDFs need OCR, which is not included. Complex table and layout interpretation is limited.

Dynamic Chunking

Chunking divides extracted text into smaller passages. The starter uses sentence-aware, word-limit-based

chunking with overlap. Overlap helps preserve context near boundaries. A more advanced dynamic chunker could use headings, paragraph and table boundaries, document type, and section structure to choose chunk boundaries.

Technology Stack

- Python — application logic.
- Streamlit — local web interface.
- pypdf — PDF text extraction.
- python-docx — DOCX text extraction.
- scikit-learn — TF-IDF and cosine similarity.ss
- Python text reading — TXT files.

System Workflow

Start the Streamlit application.